

Escola Superior de Tecnologia e Gestão de Águeda

Sistemas Operativos

Técnico Superior Profissional em Redes e Sistemas Informáticos

Introdução à Utilização de Sistemas Unix

Professor: Ivan Miguel Pires

2025/2026

OBJETIVOS DE APRENDIZAGEM

- Compreender a história e evolução do Unix/Linux
- Dominar operações básicas com ficheiros e diretórios
- Gerir permissões de acesso (chmod, chown, chgrp)
- Utilizar a shell bash e suas funcionalidades avançadas
- Controlar processos e jobs em Unix/Linux
- Conhecer ferramentas modernas de linha de comandos
- Aplicar conceitos modernos: eBPF, containers, segurança

#!/bin/bash
script bash

do {

} while (.)

HISTÓRIA DO UNIX/LINUX

- Unix (década de 70, Bell Labs)
 - Multi-utilizador, desenvolvido em C
 - Base para sistemas modernos
- Linux (1991, Linus Torvalds)
 - Versão gratuita e open-source
 - Kernel mais utilizado em servidores (>90%)
- Projeto GNU e Free Software Foundation
 - Compiladores (GCC), utilitários, bibliotecas
- Distribuições modernas (2024-2026)
 - Ubuntu, Fedora, Debian, RHEL, Arch, NixOS

AMBIENTE LINUX MODERNO

- Sessões múltiplas e simultâneas
 - Alt+Fn para alternar entre sessões
 - Ctrl+Alt+F1 para modo texto
- Ambientes gráficos modernos
 - Wayland (substituto do X11)
 - GNOME, KDE Plasma, XFCE
- Terminais modernos (2024-2026)
 - Alacritty (GPU-accelerated)
 - WezTerm, Kitty (performance otimizada)
 - Terminal emulators com suporte a imagens

GESTÃO DE PASSWORDS E AUTENTICAÇÃO

- Boas práticas tradicionais
 - Mínimo 8 caracteres (hoje: 12-16 recomendados)
 - Mistura de letras, números, símbolos
- Autenticação moderna (2024-2026)
 - Multi-factor authentication (MFA/2FA)
 - Biometria (fingerprint, face recognition)
 - Hardware tokens (YubiKey, FIDO2)
- Gestores de passwords
 - KeePassXC, Bitwarden, 1Password
 - Integração com PAM (Pluggable Authentication Modules)

FICHEIROS E DIRETÓRIOS: CONCEITOS BÁSICOS

- Nomes de ficheiros (até 256 caracteres)
- Extensões não obrigatórias (ao contrário do Windows)
- Ficheiros ocultos começam com '.' (.bashrc, .config)
- Estrutura hierárquica em árvore
 - Raiz única: /
 - Caminhos absolutos: /home/user/file.txt
 - Caminhos relativos: ../dir/file.txt
- Caracteres especiais
 - ~ (home do utilizador)
 - . (diretório atual)
 - .. (diretório pai)

COMANDOS TRADICIONAIS DE FICHEIROS

- cp - copiar ficheiros
 - cp -r dir1 dir2 (recursivo)
- mv - mover/renomear ficheiros
- rm - remover ficheiros
 - rm -rf dir (recursivo, forçado)
- cat - visualizar conteúdo
- more/less - visualização paginada
- ls - listar conteúdo
 - ls -la (detalhado + ocultos)
- cd - mudar diretório
- pwd - mostrar diretório atual
- mkdir/rmdir - criar/remover diretórios

FERRAMENTAS MODERNAS DE LINHA DE COMANDOS

- Substituições modernas de comandos Unix clássicos:
 - ripgrep (rg) vs grep
 - Até 10× mais rápido, ignora .gitignore por padrão
 - fd vs find
 - Sintaxe mais simples, execução paralela
 - bat vs cat
 - Syntax highlighting, numeração de linhas, Git integration
 - exa/eza vs ls
 - Cores, ícones, Git status, tree view
 - zoxide vs cd
 - Jump inteligente baseado em histórico
- Adoção: Ferramentas modernas ganham tração em DevOps

PERMISSÕES DE FICHEIROS EM UNIX

- Três níveis de permissões:
 - User (u) - proprietário
 - Group (g) - grupo
 - Others (o) - outros utilizadores
- Três tipos de acesso:
 - Read (r) - leitura
 - Write (w) - escrita
 - Execute (x) - execução
- Visualização: `ls -l`
 - `-rwxr-xr-x (755)`
 - `drwxr-xr-x (diretório com 755)`
- Permissões especiais: setuid, setgid, sticky bit

ALTERAR PERMISSÕES: CHMOD

- **Modo simbólico:**
 - `chmod u+x file` (adiciona execução ao user)
 - `chmod go-w file` (remove escrita de group/others)
 - `chmod a=r file` (todos apenas leitura)
- **Modo octal (numérico):**
 - `chmod 755 file` (`rxwxr-xr-x`)
 - `chmod 644 file` (`rw-r--r--`)
 - `chmod 700 file` (`rxwx-----`)
- Recursivo: `chmod -R 755 directory`
- Outros comandos:
 - `chown` - alterar proprietário
 - `chgrp` - alterar grupo

INTERPRETADOR DE COMANDOS: A SHELL

- Função da shell:
 - Interface entre utilizador e sistema operativo
 - Interpreta comandos e executa programas
- Tipos de shells:
 - `bash (Bourne Again Shell)` - padrão
 - `zsh (Z Shell)` - features avançadas
 - `fish (Friendly Interactive Shell)` - user-friendly
 - `nushell` - estruturado, moderno (2024-2026)
- Estrutura de comando:
 - `comando [opções] [argumentos]`
 - Exemplo: `ls -la /home`

↳ mostra ficheiros ocultos e permissões

SHELLS MODERNAS E PRODUTIVIDADE

- zsh (Z Shell)
 - Auto-sugestões, correção ortográfica
 - Plugins via Oh My Zsh
- fish (Friendly Interactive Shell)
 - Syntax highlighting em tempo real
 - Auto-sugestões baseadas em histórico
- nushell (Nu Shell)
 - Dados estruturados (tabelas, JSON)
 - Pipeline moderno com tipos
- Starship prompt
 - Prompt customizável, rápido, multi-shell
 - Mostra Git status, versões de linguagens

FICHEIROS DE CONFIGURAÇÃO DA BASH

- Executados no login:
 - /etc/profile (sistema)
 - ~/.bash_profile, ~/.bash_login, ~/.profile
- Executados em sessões não-login:
 - ~/.bashrc
- Executados no logout:
 - ~/.bash_logout
- Configurações modernas:
 - ~/.config/ (XDG Base Directory)
 - Dotfiles geridos com Git
 - Ferramentas: GNU Stow, chezmoi, yadm
- Personalização: aliases, funções, variáveis

METACARACTERES DA SHELL

- Para especificar conjuntos de ficheiros:
 - `*` - qualquer conjunto de caracteres
 - `?` - um único carácter
 - `[abc]` - um dos caracteres a, b ou c
 - `{jpg,png,gif}` - expansão de chaves
- Para não interpretar:
 - `\` - escapa o carácter seguinte
 - `' '` - aspas simples (literal)
 - `" "` - aspas duplas (permite \$, `, \)
- Exemplos:
 - `ls *.txt` (todos os .txt)
 - `rm file?.log` (file1.log, file2.log, etc)

VARIÁVEIS DE AMBIENTE

- Definição e uso:
 - VAR=valor (definir)
 - echo \$VAR (visualizar)
 - export VAR (exportar para subprocessos)
 - unset VAR (remover)
- Variáveis importantes:
 - HOME - diretório home do utilizador
 - PATH - caminhos de pesquisa de comandos
 - PS1 - prompt primário
 - SHELL - shell atual
 - USER - nome do utilizador
- Listar: env, printenv, set

REDIREÇÃO DE ENTRADA E SAÍDA

- Descritores de ficheiros:
 - 0 - `stdin` (entrada padrão)
 - 1 - `stdout` (saída padrão)
 - 2 - `stderr` (saída de erros)
- Operadores de redireção:
 - `<` - redireciona entrada
 - `>` - redireciona saída (sobrescreve)
 - `>>` - redireciona saída (acrescenta)
 - `2>` - redireciona erros
 - `&>` - redireciona stdout e stderr
- Pipes: `|`
 - `comando1 | comando2` (saída de `cmd1` → entrada de `cmd2`)

FACILIDADES E PRODUTIVIDADE

- Tab completion:
 - Completa comandos, ficheiros, variáveis
- Histórico de comandos:
 - `history` - listar histórico
 - `!!` - último comando
 - `!n` - comando número n
 - `Ctrl+R` - pesquisa reversa
 - Setas `↑↓` - navegar no histórico
- Atalhos de teclado:
 - `Ctrl+C` - cancelar comando
 - `Ctrl+Z` - suspender processo
 - `Ctrl+D` - logout/EOF
 - `Ctrl+L` - limpar ecrã

PROCESSOS E GESTÃO

- Conceitos:
 - Processo - programa em execução
 - PID - Process ID (identificador único)
 - PPID - Parent Process ID
- Comando ps:
 - `ps` - processos do utilizador
 - `ps aux` - todos os processos
 - `ps -ef` - formato completo
- Execução em background:
 - comando `&` (lança em background)
- Terminar processos:
 - `kill PID` (envia sinal TERM)
 - `kill -9 PID` (força terminação)

MONITORIZAÇÃO MODERNA DE PROCESSOS (2024-2026)

- Ferramentas tradicionais:
 - `top`, `htop` - monitorização interativa
- Ferramentas modernas:
 - `btpp++` - interface moderna, GPU monitoring
 - `bottom` (`btm`) - escrito em `Rust`, gráficos
 - `glances` - multiplataforma, Web UI
- Observabilidade com eBPF (2024-2026):
 - `bpfftrace` - tracing de sistema com eBPF
 - BCC tools - performance analysis
 - Overhead <5% em produção
- Métricas: eBPF permite `tracing sem modificar kernel`

eBPF: Extended Berkeley Packet Filter

- O que é eBPF (2024-2026):
 - Programas verificados executados no kernel
 - Sem necessidade de módulos kernel
 - Seguro e eficiente
- Casos de uso:
 - Observabilidade: tracing, profiling
 - Networking: filtragem de pacotes, load balancing
 - Segurança: syscall filtering, anomaly detection
- Performance (dados 2024-2026):
 - **Overhead <5%** em monitorização de rede
 - DeSFAM: <1% overhead, 94% precisão
 - KubeRosy: ~722ns para syscall filtering
 - Ferramentas: Cilium, Falco, Pixie, Tetragon

CONTROLO DE JOBS NA SHELL

- Jobs vs Processos:
 - Job - comando gerido pela shell
 - Processo - entidade do sistema operativo
- Comandos de controlo:
 - jobs - listar jobs
 - bg %n - continuar job n em background
 - fg %n - trazer job n para foreground
- Sinais de controlo:
 - Ctrl+Z - suspender processo (SIGTSTP)
 - Ctrl+C - terminar processo (SIGINT)
- Exemplo:
 - vim file.txt → Ctrl+Z → bg %1 → fg %1

FILTROS UNIX: PROCESSAMENTO DE TEXTO

- Conceito:
 - Leem entrada, processam, escrevem saída
 - Combinam-se com pipes
- Filtros tradicionais:
 - **grep** - procurar padrões
 - **wc** - contar linhas/palavras/caracteres
 - **sort** - ordenar
 - **uniq** - remover duplicados
 - **cut** - extrair colunas
 - **sed** - editor de stream
 - **awk** - processamento de texto programável
- Exemplo: `cat file.txt | grep error | wc -l`

GREP E EXPRESSÕES REGULARES

- Comandos grep:
 - `grep` padrão file (expressão regular básica)
 - `egrep` padrão file (expressão regular estendida)
 - `fgrep` palavra file (literal, sem regex)
- Metacaracteres básicos:
 - `^` - início de linha
 - `$` - fim de linha
 - `.` - qualquer caracter
 - `*` - zero ou mais ocorrências
 - `[abc]` - qualquer de a, b ou c
- Extensões egrep:
 - `+` - uma ou mais ocorrências
 - `?` - zero ou uma ocorrência
 - `|` - alternativa (ou)

RIPGREP (RG): GREP DO FUTURO

- Vantagens sobre grep tradicional:
 - Até 10× mais rápido
 - Ignora .gitignore automaticamente
 - Pesquisa recursiva por padrão
 - Syntax highlighting
 - Suporte a tipos de ficheiros
- Exemplos:
 - `rg 'pattern'` (pesquisa recursiva)
 - `rg -t py 'def'` (apenas ficheiros Python)
 - `rg -i 'error'` (case-insensitive)
- Escrito em Rust:
 - Performance e segurança de memória
 - Adoção crescente em DevOps e desenvolvimento

MANUAL DO UNIX: MAN

- Estrutura das páginas **man**:
 - NAME - nome e descrição breve
 - SYNOPSIS - sintaxe
 - DESCRIPTION - descrição detalhada
 - OPTIONS - opções disponíveis
 - EXAMPLES - exemplos
 - SEE ALSO - comandos relacionados
- Secções do manual:
 - 1 - Comandos de utilizador
 - 2 - System calls
 - 3 - Funções de biblioteca
 - 5 - Formatos de ficheiros
 - 8 - Comandos de administração
- Alternativas modernas: tldr, cheat.sh

SISTEMAS DE FICHEIROS MODERNOS (2024-2026)

- Sistemas de ficheiros tradicionais:
 - ext4 - padrão Linux
 - XFS - alto desempenho
- Sistemas modernos copy-on-write:
 - btrfs - snapshots, RAID, compressão
 - ZFS - integridade de dados, deduplicação
 - bcachefs - novo, promissor (kernel 6.7+)
- Funcionalidades modernas:
 - Snapshots instantâneos
 - Compressão transparente
 - Checksums para integridade
 - RAID nativo
- Tendência: CoW filesystems para containers/VMs

CONTAINERS E O KERNEL LINUX

- Tecnologias base de containers:
 - Namespaces - isolamento de recursos
 - cgroups - controlo de recursos
 - Capabilities - privilégios granulares
 - seccomp - filtragem de syscalls
- Performance (2024-2026):
 - bypass4netns: >30× throughput em rootless
 - SPRIGHT: ~10× menos CPU vs DPDK
 - eProbe: +0.03-0.26% memória, +2.48% latência
- Runtimes modernos:
 - Docker, containerd, CRI-O
 - Podman (rootless por padrão)
 - Segurança: eBPF para monitoring e enforcement

SEGURANÇA EM LINUX (2024-2026)

- LSM - Linux Security Modules:
 - SELinux - Mandatory Access Control
 - AppArmor - perfis de aplicações
 - eBPF LSM - políticas dinâmicas
- Syscall filtering:
 - seccomp-bpf - filtragem de system calls
 - KubeRosy: filtragem dinâmica (~722ns overhead)
- Detecção de anomalias (2024-2026):
 - DeSFAM: 94% precisão, 90% recall, <1% overhead
 - ML + eBPF para detecção em tempo real
- Hardening:
 - Kernel lockdown mode
 - Secure Boot, TPM 2.0
 - Address Space Layout Randomization (ASLR)

io_uring: Nova Interface de I/O

- O que é io_uring (kernel 5.1+):
 - Interface assíncrona de I/O
 - Shared ring buffers entre kernel e userspace
 - Elimina context switches excessivos
- Vantagens:
 - Redução de 50-60% em latência de I/O
 - Maior throughput em workloads intensivos
 - Suporte a operações de rede e ficheiros
- Adoção (2024-2026):
 - Envoy proxy: ~10% aumento bandwidth
 - Bases de dados: PostgreSQL, RocksDB
 - EXO: virtio-blk otimizado com eBPF
- Casos de uso: servidores web, DBs, storage

RUST NO KERNEL LINUX (2024-2026)

- Rust-for-Linux iniciativa:
 - Linguagem memory-safe no kernel
 - Previne buffer overflows, use-after-free
 - Kernel 6.1+ com suporte inicial
- Benefícios:
 - Segurança de memória garantida em compile-time
 - Redução de vulnerabilidades
 - Manutenção mais fácil de código complexo
- Estado atual (2024-2026):
 - Drivers experimentais em Rust
 - Ferramentas e bindings em desenvolvimento
 - Overhead <9% em protótipos (rOOM)
- Futuro: Mais subsistemas migrados para Rust

LINUX CLOUD-NATIVE (2024-2026)

- Distribuições otimizadas para cloud:
 - Flatcar Container Linux
 - Bottlerocket (AWS)
 - Talos Linux (Kubernetes-native)
- Características:
 - Imutáveis (immutable infrastructure)
 - Mínimas (apenas o necessário)
 - Atualizações atômicas
 - Boot rápido (<5s)
- Kubernetes e eBPF:
 - Cilium: networking com eBPF
 - eZtunnel: >20% redução latência service mesh
 - Tendência: Kernel customizado para workloads específicos

MULTIPLEXERS E PRODUTIVIDADE

- **tmux (Terminal Multiplexer):**
 - Múltiplas sessões num terminal
 - Sessões persistentes (sobrevivem a desconexões)
 - Split panes, windows
- Alternativas modernas:
 - zellij - escrito em Rust, UI moderna
 - Configuração mais simples que tmux
 - Layouts automáticos
- Casos de uso:
 - Desenvolvimento remoto
 - Administração de servidores
 - Pair programming
- Integração com shells modernas e Starship prompt

ECOSSISTEMA DE DESENVOLVIMENTO (2024-2026)

- Editores e IDEs:
 - Neovim - Vim moderno com LSP
 - VS Code - dominante em desenvolvimento
 - Helix - editor moderno em Rust
- Controlo de versões:
 - Git - padrão da indústria
 - lazygit - TUI para Git
 - GitHub CLI (gh)
- Ferramentas de build:
 - Make, CMake (tradicionais)
 - Meson, Ninja (modernos, rápidos)
- Debugging:
 - gdb - debugger tradicional
 - lldb - debugger LLVM
 - bpftrace - debugging com eBPF

TENDÊNCIAS FUTURAS EM UNIX/LINUX

- Kernel e sistema:
 - Mais Rust no kernel (segurança)
 - eBPF como standard de extensibilidade
 - io_uring adoção generalizada
- Containers e orquestração:
 - Kubernetes como 'OS distribuído'
 - WebAssembly para isolamento leve
 - Unikernels para serverless
- Segurança:
 - Zero-trust por padrão
 - Verificação formal (seL4)
 - Hardware security (ARM TrustZone, Intel SGX)
- User experience:
 - CLIs mais user-friendly
 - TUIs (Terminal UIs) vs GUIs

RECURSOS PARA APRENDIZAGEM

- Documentação oficial:
 - man pages (man comando)
 - info pages (info comando)
 - kernel.org - documentação do kernel
- Recursos online modernos:
 - tldr.sh - exemplos práticos de comandos
 - explainshell.com - explica comandos shell
 - cheat.sh - cheat sheets interativos
- Livros recomendados:
 - The Linux Programming Interface (Michael Kerrisk)
 - Linux Kernel Development (Robert Love)
- Prática:
 - Laboratórios práticos essenciais
 - Projetos pessoais, contribuir para open-source

UNIX TRADICIONAL VS MODERNO (2024-2026)

- Comandos:
 - `grep` → `ripgrep` (10× mais rápido)
 - `find` → `fd` (sintaxe simples)
 - `cat` → `bat` (syntax highlighting)
 - `ls` → `eza` (ícones, Git status)
- Shells:
 - `bash` → `zsh/fish` (auto-sugestões)
- Monitorização:
 - `top` → `btob++/bottom` (interfaces modernas)
- I/O:
 - `read/write syscalls` → `io_uring` (50-60% latência)
- Observabilidade:
 - `strace` → `eBPF/bpftrace` (<5% overhead)
- Filosofia mantida: pequenas ferramentas, pipes, texto

CASOS DE ESTUDO MODERNOS

- DeSFAM (Container Security):
 - eBPF + ML para detecção de anomalias
 - 94% precisão, <1% overhead
- bypass4netns (Rootless Containers):
 - >30× throughput em containers rootless
- eZtunnel (Service Mesh):
 - >20% redução latência com eBPF
- KubeRosy (Dynamic Syscall Filtering):
 - ~722ns overhead vs seccomp estático
- SPRIGHT (Serverless):
 - ~10× menos CPU que DPDK
- Lição: eBPF é tecnologia chave em inovações Linux

CONCLUSÕES

- Unix/Linux: Sistema robusto e em evolução
- Conceitos fundamentais mantêm-se:
 - Ficheiros, processos, permissões, shell
- Inovações modernas (2024-2026):
 - eBPF - extensibilidade segura
 - io_uring - I/O de alta performance
 - Rust - segurança de memória
 - Containers - isolamento e portabilidade
- Ferramentas CLI modernas melhoram produtividade
- Segurança é prioridade crescente
- Linux domina: servidores, cloud, IoT, embedded
- Aprendizagem contínua essencial

QUESTÕES?